IMAGE VIDEO PROCESSING

Project Report

JEWELFX STUDIO: REAL-TIME VIRTUAL JEWELLERY TRY-ON USING FACIAL LANDMARKS



SUBMITTED BY:

Jaiveer Sethi

Manshay Bamboria

Dhwani Jain

Class / Batch: BTech CE A1

ACKNOWLEDGEMENT

I would like to thank my subject faculty and project guide for the opportunity to work on a practical application of Image and Video Processing. This project helped me connect concepts such as video acquisition, landmark-based geometry, coordinate transformation, temporal smoothing and image compositing with an interactive application.

I am also thankful to the developers and maintainers of the open-source tools and documentation used during development, particularly MediaPipe and the web platform APIs. I appreciate the feedback and encouragement received from classmates, friends and family during the preparation of the prototype and report.

ABSTRACT

JewelFX Studio is a browser-based virtual jewellery try-on prototype developed as an application of Image and Video Processing. The system uses a webcam to obtain a continuous video stream and applies facial landmark detection to estimate reference points on the face. Stylised two-dimensional jewellery graphics—earrings, a necklace, a nose ring and a maang tikka—are positioned over the live preview. Users can switch individual items on or off, select gold, silver or rose-gold colour themes, adjust overlay size and tracking smoothness, capture a still image, and record a short video where supported by the browser.

The application does not require a project-specific training dataset. It uses a pre-trained face-landmark model supplied by MediaPipe and procedural jewellery graphics. The main stages are camera acquisition, landmark inference, coordinate mapping, temporal smoothing, canvas rendering and compositing. The application runs in a modern browser and requires permission to access the camera.

This is an educational prototype rather than a production-grade augmented-reality system. Placement is approximate, particularly for the necklace, which uses face-width and chin-based geometry instead of explicit neck segmentation. Results depend on lighting, camera position, head pose and browser performance. No unmeasured benchmark results are claimed in this report.

Keywords: Image processing, video processing, augmented reality, facial landmarks, MediaPipe, canvas compositing, real-time tracking, virtual try-on.

TABLE OF CONTENTS

Acknowledgement

Abstract

Abbreviations

Chapter 1 – Introduction

Chapter 2 – Problem Definition and Objectives

Chapter 3 – Background and Related Concepts

Chapter 4 – System Requirements and Feasibility

Chapter 5 – System Design and Architecture

Chapter 6 – Methodology and Algorithms

Chapter 7 – Implementation Details

Chapter 8 – Testing and Results

Chapter 9 – Limitations and Future Scope

Chapter 10 – Conclusion

References

Appendix A – User Manual

ABBREVIATIONS

Term	Meaning
AR	Augmented Reality
FPS	Frames Per Second
GUI	Graphical User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
PNG	Portable Network Graphics
WebM	Web Media video container

CHAPTER 1 – INTRODUCTION

1.1 Background

Image and Video Processing involves acquiring visual data, transforming it, extracting information and presenting the result. Many applications perform these operations continuously on camera frames, including video-conferencing effects, augmented-reality shopping and interactive photography.

Jewellery is a useful setting for an augmented-reality demonstration because an accessory must remain visually connected to the user as the head moves. A virtual try-on system can give a quick impression of a design without requiring the person to physically wear it.

1.2 Project overview

JewelFX Studio is a browser-based prototype that overlays four jewellery types on a live camera preview: jhumka-style earrings, a Rani Haar-style necklace, a nath nose ring and a maang tikka. Users can change the metal colour theme, alter overall scale, adjust tracking smoothness, enable or disable each accessory, capture a PNG image and record a WebM video when supported.

1.3 Motivation

The project turns classroom concepts into an interactive application without preparing a labelled training dataset. A pre-trained model provides facial landmarks, while conventional drawing and compositing operations place the jewellery graphics.

1.4 Scope

- Runs in a modern desktop browser and uses the webcam with permission.
- Supports four stylised jewellery overlays and three colour themes.
- Demonstrates landmark tracking, geometric placement, smoothing and canvas rendering.
- Supports local photo capture and browser-dependent video recording.
- It is not designed for jewellery measurement, authenticity verification or commercial-grade fitting.

CHAPTER 2 – PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem statement

Trying several jewellery designs physically can take time and may require access to the actual products. This project explores whether a lightweight browser application can create an immediate visual preview by placing jewellery graphics on live face video. The challenge is to keep overlays reasonably aligned as the face moves while maintaining a responsive interface.

2.2 Objectives

- Acquire a live video stream through the browser camera API.
- Detect facial landmarks using a pre-trained model without training a new dataset.
- Estimate anchor positions for earrings, necklace, nose ring and maang tikka.
- Apply temporal smoothing to reduce sudden changes in overlay position.
- Render jewellery graphics with colour themes and adjustable scale.
- Provide camera controls, photo capture and video recording.
- Measure FPS and frame-processing time during practical testing.
- Document requirements, algorithms, limitations and improvements.

Objective	Evaluation method
Camera acquisition	Camera starts with permission and stops cleanly.
Landmark tracking	Face detected and overlays update with movement.
Filter controls	Toggles and colour options alter the preview.
Smoothing	Compare jitter and lag at different settings.
Capture	PNG downloads and opens.
Recording	WebM is produced where supported.
Performance	Record observed FPS and processing time.

2.3 Expected outcome

The expected outcome is a working prototype demonstrating a complete visual-processing pipeline. Evaluation should use tests performed on the target laptop rather than assumed performance figures.

2.4 Constraints and assumptions

- A webcam is available and browser permission is granted.
- The browser can download the MediaPipe runtime and model on first use.
- The face is reasonably visible and lighting is adequate.
- Accessories are stylised 2D overlays, not physically accurate 3D objects.
- The system does not train or fine-tune a model on a custom dataset.

CHAPTER 3 – BACKGROUND AND RELATED CONCEPTS

3.1 Digital video frames

A video stream can be treated as a sequence of still images captured at regular intervals. A real-time application reads a frame, processes it and draws the output repeatedly. If processing takes too long, the preview may appear delayed or less smooth. Frame rate is commonly expressed in frames per second (FPS).

3.2 Facial landmark detection

Facial landmark detection estimates characteristic points such as eye corners, nose, chin and face outline. A pre-trained model makes it possible to locate approximate anchors without collecting and labelling a project-specific dataset. Normalised coordinates are converted to pixels using x_pixel = x_normalised × image_width and y_pixel = y_normalised × image_height.

3.3 Coordinate mapping

The overlay must use the same coordinate system as the camera frame. Coordinates are scaled from normalised values to canvas pixels. Accessory dimensions are scaled using an estimate of face width or another distance between landmarks. Rotation can be estimated from landmark angles, although some placements in this prototype are simplified.

3.4 Temporal smoothing

Landmark estimates can fluctuate between frames. Exponential smoothing reduces jitter: $p_s(t) = \alpha\, p_s(t-1) + (1-\alpha)\, p(t)$, where $p(t)$ is the current point and α controls the weight assigned to the previous smoothed position. Higher α gives steadier but potentially delayed movement; lower α follows movement more quickly.

3.5 Canvas rendering and compositing

HTML Canvas draws lines, curves, circles and custom shapes. Jewellery graphics use gradients, strokes and polygons. For transparent graphics, alpha compositing combines foreground and background pixels: $C = \alpha F + (1-\alpha)B$, where F is foreground colour, B is background colour and α is opacity.

3.6 Tools and dataset

MediaPipe provides face landmarking. getUserMedia provides camera access, requestAnimationFrame supports repeated rendering, Canvas draws overlays, and MediaRecorder can capture a canvas stream. JewelFX does not require a custom training dataset, although the pre-trained runtime and model are downloaded from external hosting on first use.

CHAPTER 4 – SYSTEM REQUIREMENTS AND FEASIBILITY

4.1 Hardware requirements

Component	Minimum practical requirement	Recommended
Computer	Laptop/desktop with modern browser	Recent multi-core CPU
Camera	Built-in or USB webcam	720p or better
Memory	4 GB RAM	8 GB or more
Display	1280 × 720	Full HD
Internet	Initial model/runtime download	Stable connection

4.2 Software requirements

Software	Purpose
Windows/macOS/Linux	Host operating system
Chrome/Edge or compatible browser	Run app and access camera
Python 3	Simple local HTTP server
MediaPipe Tasks Vision	Face landmark inference
HTML, CSS, JavaScript	Interface and application logic

4.3 Functional requirements

- Start and stop the camera.
- Show a live preview when permission is granted.
- Detect a face and position selected overlays.
- Switch jewellery types and metal colour.
- Adjust size and smoothing.
- Capture a still image and attempt video recording.
- Show tracking status, FPS and frame processing time.

4.4 Non-functional requirements

- Usability: controls should be understandable.
- Responsiveness: frame processing should be fast enough for a useful preview.
- Privacy: camera frames remain in the browser in this implementation.
- Compatibility: run from localhost or HTTPS rather than file://.
- Maintainability: separate interface, styles and processing logic.

4.5 Feasibility

The project is technically feasible for a classroom prototype because the tools are available as browser libraries and APIs. Cost is low when an existing laptop and webcam are used. Main risks include browser compatibility, model download access, lighting and imperfect overlay placement.

CHAPTER 5 – SYSTEM DESIGN AND ARCHITECTURE

5.1 High-level architecture

Stage	Input	Operation	Output
1. Video acquisition	Webcam	getUserMedia and video element	Live frames
2. Landmark inference	Current frame	MediaPipe Face Landmarker	Landmark coordinates
3. Coordinate processing	Landmarks	Pixel mapping and smoothing	Stable anchors
4. Jewellery rendering	Anchors and settings	Canvas paths, curves, gradients	Overlay
5. Preview composition	Video and overlay	Layered display	Live try-on view
6. Export	Preview stream	Canvas capture / MediaRecorder	PNG or WebM

5.2 Main modules

Module	Responsibility
Camera module	Requests permission, starts video and stops tracks.
Face tracking	Loads model and estimates landmarks.
Smoothing	Blends current and prior landmark estimates.
Jewellery renderer	Draws enabled accessories using anchor positions.
Control panel	Manages toggles, colour, scale and smoothing.
Capture module	Exports a photo or supported video recording.
Metrics	Shows tracking state, approximate FPS and processing time.

5.3 Data flow

The camera supplies frames to the video element. The tracker processes the current frame and returns facial landmarks. Coordinates are smoothed and passed to the renderer. The renderer draws enabled accessories on a transparent canvas layered over the video. User controls update rendering parameters. Capture operations combine the video frame and overlay.

5.4 Privacy

Camera permission is requested by the browser and frame processing occurs on the client. The runtime and model are downloaded from public hosting, but the app does not need to upload camera frames to a backend. Users should stop the camera when finished. A production deployment should review dependency versions, hosting and privacy notices.

CHAPTER 6 – METHODOLOGY AND ALGORITHMS

6.1 Workflow

1. Request camera access.
2. Load MediaPipe runtime and face-landmark model.
3. Read the current frame and match canvas dimensions to video dimensions.
4. Run face landmark detection.
5. Smooth landmark coordinates if a face is detected.
6. Convert selected landmarks to pixel coordinates and estimate face width.
7. Draw each enabled accessory at its approximate anchor.
8. Update FPS, tracking status and processing time.
9. Repeat until the camera is stopped.

6.2 Camera acquisition

The app uses navigator.mediaDevices.getUserMedia to request a video stream and attaches it to an HTML video element. Modern browsers generally require HTTPS or localhost and explicit user permission. Errors should be displayed if permission is denied or no camera is available.

6.3 Landmark inference and accessory placement

The Face Landmarker model estimates a collection of facial points. The prototype uses points around the face outline, chin, nose, forehead and eye region as visual anchors. Earrings are drawn near the left and right face outline, the nose ring near the nose, and the maang tikka near the central forehead. The necklace is approximated from face width and chin position, so it is less reliable than an implementation using neck and shoulder landmarks.

6.4 Scaling and smoothing

A size control multiplies accessory geometry by a user-selected scale factor. Face-relative accessories use estimated face width as a reference. For smoothing, the current point is blended with the previous smoothed point. Excessive smoothing can make overlays lag; insufficient smoothing can cause jitter.

6.5 Rendering

Canvas paths and primitive shapes draw the jewellery. Gradients suggest metallic highlights and shadows; polygons and circles represent stones and beads. Gold, silver and rose-gold themes change the rendering palette. This is a stylised effect, not a physically based material simulation.

6.6 Performance measurement

FPS is estimated by counting processed frames over a time interval. Processing time is measured around the frame-processing section. Results should be recorded on the target device under documented conditions because browser scheduling, CPU/GPU availability and camera resolution affect performance.

Operation	Purpose	Consideration
MediaPipe Face Landmarker	Estimate facial reference points	Inference time and pose robustness
Normalised-to-pixel mapping	Convert coordinates to canvas pixels	Video and canvas dimensions must match
Exponential smoothing	Reduce jitter	Higher smoothing adds lag
Canvas drawing	Render jewellery	Stylised rather than photorealistic
Layered compositing	Combine video and overlay	Overlay must be aligned
FPS timing	Estimate responsiveness	Not a full end-to-end latency measure

CHAPTER 7 – IMPLEMENTATION DETAILS

7.1 Project files

File	Description
index.html	Page structure, camera stage, controls and help dialog.
styles.css	Responsive layout, typography and colour palette.
app.js	Camera access, MediaPipe, landmark processing, drawing and export.
README.md	Setup, requirements, limitations and viva concepts.

7.2 Technologies

- HTML5 for page structure, video and canvas.
- CSS for the responsive studio interface.
- JavaScript ES modules for application logic.
- MediaPipe Tasks Vision for pre-trained face landmark detection.
- Canvas 2D API for jewellery rendering and visual layers.
- MediaRecorder and canvas captureStream for supported video recording.
- Python HTTP server for local development.

7.3 Run on Windows

10. Extract the project ZIP.
11. Open PowerShell in the folder containing index.html, app.js and styles.css.
12. Run: py -m http.server 8000
13. Keep the terminal open.
14. Open http://localhost:8000 in Chrome or Edge.
15. Click Start camera and allow permission.
16. Test jewellery controls and capture options.
17. Press Ctrl+C in PowerShell to stop the server.

7.4 Run on macOS

18. Extract the project ZIP.
19. Open Terminal and change directory to the folder containing index.html.
20. Run: python3 -m http.server 8000
21. Open http://localhost:8000 in a browser.
22. Allow camera access and test the filters.
23. Press Ctrl+C to stop the server.

7.5 Why use localhost?

Opening the page as a file can prevent camera access or module loading because browsers apply security restrictions to file URLs. A local HTTP server provides a localhost origin that modern browsers treat as a secure context.

7.6 Implementation clarification

The current browser application uses JavaScript Canvas drawing rather than the OpenCV library directly. It demonstrates image- and video-processing principles, but the report should not claim direct OpenCV integration unless OpenCV.js or another OpenCV binding is added. The jewellery graphics are procedural and do not rely on a custom dataset.

CHAPTER 8 – TESTING AND RESULTS

8.1 Testing approach

Testing should cover the interface, camera lifecycle, tracking, each accessory, settings, exports and performance. Functional tests verify whether operations work; visual tests assess overlay placement; performance tests record FPS and processing time under documented conditions.

8.2 Functional test cases

ID	Test procedure	Expected result	Actual result
TC-01	Open site on localhost	Interface loads	Fill after testing
TC-02	Start camera and allow access	Live video appears	Fill after testing
TC-03	Deny camera permission	Readable error shown	Fill after testing
TC-04	Move face left/right	Overlays follow approximately	Fill after testing
TC-05	Toggle each jewellery item	Selected item appears/disappears	Fill after testing
TC-06	Change metal finish	Palette changes	Fill after testing
TC-07	Adjust size	Overlay size changes	Fill after testing
TC-08	Adjust smoothing	Stability/lag changes	Fill after testing
TC-09	Capture photo	PNG downloads and opens	Fill after testing
TC-10	Record and stop video	WebM downloads if supported	Fill after testing
TC-11	Stop camera	Camera tracks stop	Fill after testing
TC-12	Reload app	App can start again	Fill after testing

8.3 Performance observations

No benchmark values are inserted because frame rate and alignment depend on the actual laptop, browser and camera. Complete this table after testing the final version.

8.4 Visual evaluation criteria

- Anchor accuracy: accessories remain near intended facial locations.
- Stability: overlays do not shake excessively while the head is still.
- Responsiveness: overlays follow movement without noticeable delay.
- Scale consistency: accessories resize sensibly as face size changes.
- Visual quality: jewellery remains visible and colours are distinguishable.
- Recovery: tracking resumes when the face re-enters the frame.

8.5 Results discussion

After testing, summarise which functions worked consistently, which conditions caused tracking loss, and how smoothing affected responsiveness. Add screenshots of the live preview and one test for each accessory. If a test fails, record the browser, error message and reproduction steps. Do not report expected outcomes as measured results.

8.6 Known items to verify

Necklace placement is approximate because it uses chin and face-width estimates instead of explicit neck segmentation. The nose ring and forehead ornament may need adjustment for head rotation. Recording depends on browser support for MediaRecorder and the chosen MIME type.

CHAPTER 9 – LIMITATIONS AND FUTURE SCOPE

9.1 Limitations

- Overlays are stylised 2D graphics, not realistic 3D jewellery.
- Necklace placement is approximate and may not follow the neck or shoulders.
- Large head rotations, occlusion, low light and partial faces can reduce alignment quality.
- The system does not estimate true jewellery size in millimetres.
- It does not verify gold purity, gemstone quality or manufacturing defects.
- The runtime/model is downloaded from an external host on first use.
- FPS and processing time are approximate indicators, not complete end-to-end latency.
- Video export depends on browser support.

9.2 Future improvements

- Use transparent jewellery assets or 3D models with improved texture and lighting.
- Add head-pose estimation and rotation-aware transformations.
- Estimate neck and shoulders with pose landmarks or segmentation.
- Add occlusion handling for hair, hands and facial overlap.
- Provide separate position and scale controls for each accessory.
- Allow user-uploaded transparent jewellery assets.
- Add a local screenshot gallery and product comparison.
- Host model assets locally where licensing permits.
- Conduct structured usability testing with consented participants.
- Explore WebGL or a 3D rendering library for more realistic materials.

9.3 Potential applications

A refined version could support online jewellery previews, showroom kiosks, product catalogues, social-media effects and design prototyping. Commercial use would require improved fit accuracy, clear privacy controls, accessibility testing and evaluation across diverse faces and lighting.

CHAPTER 10 – CONCLUSION

JewelFX Studio demonstrates how image and video processing concepts can be combined in a browser-based virtual jewellery try-on application. A live camera stream supplies frames, a pre-trained facial landmark model estimates reference points, and a canvas renderer places stylised earrings, a necklace, a nose ring and a maang tikka on the preview. Controls allow the user to change jewellery pieces, colour theme, size and tracking smoothness, while capture features extend the application beyond a live demonstration.

The project meets its educational goal by presenting a complete processing pipeline without requiring a custom training dataset. It also illustrates real-time vision trade-offs: smoother tracking can introduce lag, landmarks are estimates, and two-dimensional overlays cannot fully reproduce three-dimensional jewellery. Necklace placement is particularly approximate because the current version does not explicitly identify the neck region.

The next step is to test the application systematically on the target device, record actual performance, add screenshots and refine accessory placement. Pose handling, neck segmentation, occlusion and more realistic assets would improve the experience.

REFERENCES

[1] Google, “MediaPipe Face Landmarker,” MediaPipe documentation. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

[2] Google, “Face Landmarker for Web,” MediaPipe documentation. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/web_js

[3] MDN Web Docs, “MediaDevices: getUserMedia()”. https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[4] MDN Web Docs, “Canvas API”. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

[5] MDN Web Docs, “MediaRecorder”. https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder

[6] MDN Web Docs, “Window: requestAnimationFrame()”. https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

[7] WHATWG, “HTML Living Standard — The canvas element”. https://html.spec.whatwg.org/

[8] W3C, “Media Capture and Streams”. https://www.w3.org/TR/mediacapture-streams/

Add the date on which online documentation was accessed before submission. Verify the exact library version and licensing terms used in the final build.

APPENDIX A – USER MANUAL

A.1 Starting the application

24. Extract the ZIP to a known folder.
25. Open a terminal in that folder.
26. Run `py -m http.server 8000` on Windows or `python3 -m http.server 8000` on macOS.
27. Open http://localhost:8000 in a supported browser.
28. Allow camera access and wait for the tracking model to load.

A.2 Controls

- Start camera: requests access and begins processing.
- Jewellery cards: enable or disable earrings, necklace, nose ring and maang tikka.
- Metal finish: select gold, silver or rose gold.
- Jewellery size: adjust overlay scale.
- Tracking smoothness: balance stability and responsiveness.
- Capture photo: downloads a PNG preview.
- Record video: starts/stops browser-supported WebM recording.
- Stop camera: stops the stream and returns to idle.
- Reset: restores default filters and slider values.

A.3 Troubleshooting

Problem	Possible solution
Camera permission denied	Allow camera in browser site settings and reload.
localhost refuses connection	Start the server and keep the terminal open.
Directory listing appears	Run the server in the folder containing index.html.
Face not detected	Improve lighting and face the camera.
Overlays jitter	Increase smoothing slightly; reduce it if lag is noticeable.
Model does not load	Check internet and browser console for network errors.

ADDITIONAL TECHNICAL DESIGN AND EVALUATION

This section expands the report with the design artefacts, algorithm specification, risk analysis and evaluation procedure commonly expected in a full project submission. It should be read alongside the earlier chapters. Any final claims about performance must be based on tests performed on the actual system.

5.5 Architecture diagram

Figure 5.1 summarises the path from webcam acquisition to face-landmark inference, coordinate stabilisation, jewellery rendering and export. User settings feed into the rendering stage. The application is client-side: the browser handles camera frames and draws the preview.

Figure 5.1 — JewelFX Studio high-level architecture

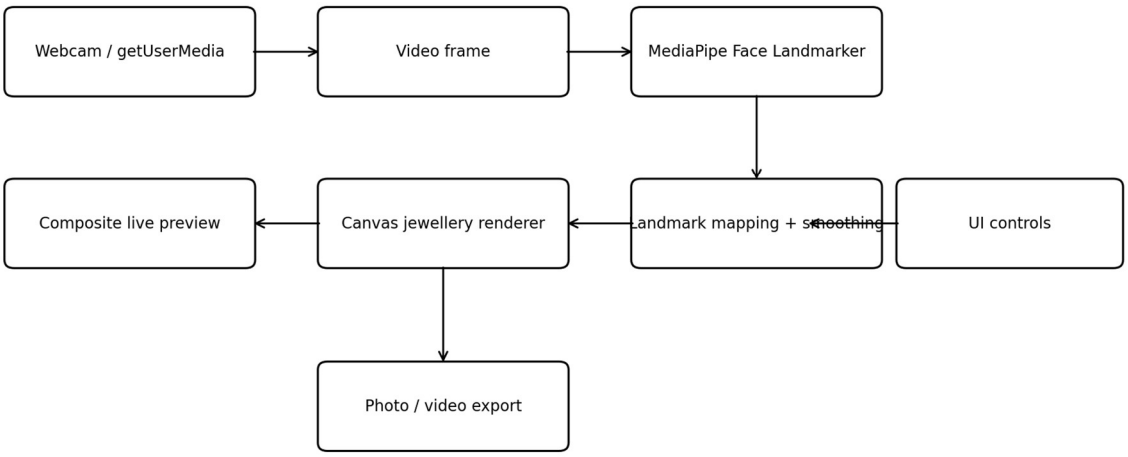


Figure 5.1: High-level architecture of JewelFX Studio.

6.9 Processing flow diagram

The application repeats the processing sequence while the camera is active. If no face is detected, the interface should avoid leaving stale jewellery in an unrelated position; the expected behaviour is to clear or hide the overlays until tracking resumes.

Figure 6.1 — Real-time processing workflow

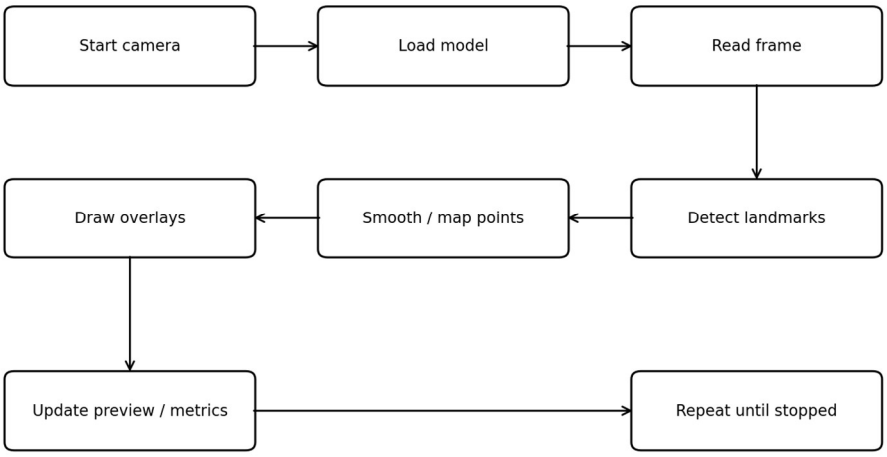


Figure 6.1: Frame-processing workflow.

6.10 Algorithm specification

Algorithm 1: Initialisation

29. 1. Read the user's camera-start action.
30. 2. Request a video stream using the browser camera API.
31. 3. Load the MediaPipe Face Landmarker runtime and model.
32. 4. Configure the video and canvas dimensions.
33. 5. Initialise the previous-smoothed-landmark state.
34. 6. Start the repeated frame-processing loop.

Algorithm 2: Frame processing

35. 1. Obtain the latest available video frame.
36. 2. Record a start timestamp for performance measurement.
37. 3. Run face-landmark inference on the frame.
38. 4. If landmarks are present, map normalised coordinates to canvas pixels.
39. 5. Smooth each selected point using the configured smoothing factor.
40. 6. Estimate face width and accessory anchor points.
41. 7. Clear the overlay canvas and draw enabled jewellery items.

42. 8. Update the preview and frame-rate statistics.
43. 9. Record elapsed processing time and schedule the next frame.

Algorithm 3: Camera shutdown

44. 1. Cancel or stop the processing loop.
45. 2. Stop every media track in the camera stream.
46. 3. Clear the video source and overlay canvas as appropriate.
47. 4. Reset tracking state and update the interface status.

6.11 Core equations

Normalised coordinate mapping:

$$x_{px} = x_{norm} \times W, \qquad y_{px} = y_{norm} \times H$$

where W and H are the current canvas width and height in pixels.

Exponential smoothing:

$$p_s(t) = \alpha\, p_s(t-1) + (1 - \alpha)\, p(t)$$

where p(t) is the current landmark estimate, $p_s(t-1)$ is the prior smoothed point, and α is a smoothing coefficient between 0 and 1.

Approximate frame rate:

$$FPS \approx \text{number of processed frames} / \text{elapsed time in seconds}$$

These equations describe the general method. The precise coefficient range and metric implementation should be checked against the submitted source code.

5.6 Use cases

Primary actor: the user operating the browser.

- Start camera and grant permission.
- View live face tracking.
- Enable or disable earrings, necklace, nose ring and maang tikka.
- Change the metal colour theme.
- Adjust accessory scale and tracking smoothness.

- Capture a still image.
- Record and download a short video where supported.
- Stop the camera.

5.7 Design decisions and rationale

Browser-based delivery. Avoids a separate desktop installation and makes the demonstration accessible through a local web server.

Pre-trained landmarks. Avoids collecting a custom training dataset and keeps the project focused on video processing and visualisation.

Procedural Canvas graphics. Makes colour changes, scaling and drawing adjustments straightforward without requiring jewellery asset files.

Client-side processing. Avoids sending camera frames to a server in the basic prototype and reduces backend complexity.

2D overlay approach. Keeps computation relatively lightweight, while accepting that perspective, occlusion and realistic depth are limited.

8.7 Detailed evaluation protocol

Use the following procedure so results are repeatable:

48. Record the laptop model, operating system, browser version, camera resolution and whether other heavy applications are open.
49. Use a fixed camera position and a consistent face-to-camera distance for baseline measurements.
50. Run each test for at least 30 seconds after the model has loaded, so initial download and warm-up time do not distort the observation.
51. Record approximate FPS and processing time with no overlays, then with all four overlays enabled.
52. Repeat under bright lighting, dim lighting, slow head turns and moderate left/right head rotation.
53. Record tracking loss, visible jitter, delay, and whether the overlays recover when the face returns to the frame.

54. Run photo capture and video recording tests separately and open the resulting files to verify them.
55. Report the actual observations, including failures; do not replace missing results with estimated numbers.

8.8 Risk analysis

Risk	Effect	Mitigation
Camera permission denied	No live preview	Explain browser permission steps and allow retry.
Model/network download fails	Tracking never starts	Show load status; check network and model URL.
Wrong working directory	Directory listing instead of app	Start server from folder containing index.html.
Port 8000 already in use	Server fails to bind	Stop old server or use another port and matching URL.
Low light / head rotation	Landmarks unstable or lost	Improve lighting; explain supported pose range.
Necklace curve misplaced	Poor visual realism	Tune anchor geometry or add neck/shoulder landmarks.
Unsupported video MIME type	Recording fails	Check MediaRecorder support and select supported format.

8.9 Acceptance criteria

The project may be considered ready for a classroom demonstration when the website opens from localhost, camera permission works, the four controls can be tested independently, colour and size settings visibly affect the overlays, the camera can be stopped cleanly, and capture/recording features either work or display a clear browser-support message. Visual alignment and performance should be reported separately from functional success.

9.4 Ethical, privacy and accessibility considerations

The application processes a live face image, so the user should be told when the camera is active and should retain control over starting and stopping it. The basic design does not require server-side storage of video. Screenshots or recordings should only be saved or shared with the subject's

permission. The interface should provide readable labels, visible focus states and error messages that do not rely on colour alone. Testing should include different lighting conditions and a range of face shapes and skin tones, while avoiding claims of universal accuracy from a small informal test.